

Koneksi Database dengan Spring Boot

Spring Data JPA • Hibernate • MySQL

Tujuan Memahami koneksi Spring Boot ke database relasional	Durasi 1,5 Jam (90 Menit)	Tools IntelliJ / VS Code • MySQL • Postman
Link Code	https://github.com/halofren/KelasPBO/tree/main/data	

Tujuan Pembelajaran (Learning Objectives)

1. Memahami konsep arsitektur layer Spring Boot dan peran masing-masing komponen
2. Mampu mengkonfigurasi koneksi database melalui application.properties
3. Mampu membuat Entity class dengan anotasi JPA yang tepat
4. Mampu membuat Repository dan memanfaatkan Spring Data JPA
5. Mampu membangun CRUD API lengkap dengan Service dan Controller
6. Mampu mendiagnosis dan menyelesaikan error umum koneksi database

Agenda Pembelajaran

Topik	Metode
Pembukaan & Tujuan Pembelajaran	Presentasi
Konsep Dasar: Arsitektur & Teknologi	Presentasi + Diskusi
Persiapan Project (Spring Initializr)	Demo Live
Konfigurasi Koneksi Database	Demo + Hands-on
Entity & Repository Layer	Demo + Hands-on
Service & Controller Layer	Demo + Hands-on
Demo CRUD Lengkap + Testing Postman	Demo + Hands-on
Error Umum & Tips Debug	Diskusi
Ringkasan & Tanya Jawab	Tanya Jawab

1.1 Apa itu Spring Boot?

Spring Boot adalah framework open-source berbasis Java yang dibangun di atas Spring Framework. Tujuan utamanya adalah menyederhanakan proses setup dan konfigurasi sehingga developer bisa langsung fokus menulis logika bisnis.

Fitur unggulan Spring Boot:

- Auto-configuration — konfigurasi otomatis berdasarkan dependensi yang ditambahkan
- Embedded Server — sudah termasuk Tomcat/Jetty, tidak perlu deploy terpisah
- Starter Dependencies — bundle dependensi siap pakai (spring-boot-starter-*)
- Actuator — monitoring dan management endpoint built-in
- Spring Initializr — project generator berbasis web di start.spring.io

1.2 Perbandingan Teknologi Akses Database

Teknologi	Level Abstraksi	Konfigurasi	Keterangan
JDBC	Rendah	Manual & verbose	Query SQL murni, koneksi dikelola sendiri
JPA	Tinggi	Spesifikasi Java	Standar ORM Java, bukan implementasi
Hibernate	Tinggi	Sedang	Implementasi JPA paling populer
Spring Data JPA	Sangat Tinggi	Minimal	Abstraksi Hibernate, yang kita gunakan

1.3 Arsitektur Layer Aplikasi

Spring Boot menerapkan arsitektur berlapis (layered architecture) yang memisahkan tanggung jawab setiap komponen:

Layer	Anotasi	Tanggung Jawab	Contoh
Presentation	@RestController	Terima HTTP request, kirim response	UserController.java
Business	@Service	Logika bisnis & validasi data	UserService.java
Data Access	@Repository	Operasi database (CRUD)	UserRepository.java
Data	@Entity	Representasi tabel database	User.java

Layer Arsitektur Aplikasi



→ Request masuk ke Controller → diteruskan ke Service → Service panggil Repository → Repository akses Database

1.4 Apa itu ORM?

ORM (Object Relational Mapping) adalah teknik pemetaan antara objek Java (class) dengan tabel di database relasional. Dengan ORM, kita bisa berinteraksi dengan database menggunakan objek Java tanpa menulis SQL secara langsung.

Java / ORM	Database Relasional
Class	Tabel
Object	Baris (Row)
Field / Attribute	Kolom (Column)
Constructor	INSERT
Getter/Setter	SELECT / UPDATE

2.1 Membuat Project dengan Spring Initializr

Spring Initializr adalah tool berbasis web untuk membuat project Spring Boot baru secara instan.

URL

<https://start.spring.io>

Langkah-langkah membuat project:

1. Buka <https://start.spring.io> di browser
2. Pilih: Project = Maven, Language = Java, Spring Boot = 4.x.x
3. Isi Metadata: Group = com.tup, Artifact = data, Name = data
4. Packaging = Jar, Java = 17 atau 21
5. Klik ADD DEPENDENCIES dan tambahkan dependensi berikut
6. Klik GENERATE → file ZIP akan terunduh
7. Ekstrak ZIP dan buka dengan IntelliJ IDEA atau VS Code

2.2 Dependensi yang Dibutuhkan

Dependensi	Artifact ID	Fungsi
Spring Web	spring-boot-starter-web	Membuat REST API dengan Spring MVC
Spring Data JPA	spring-boot-starter-data-jpa	ORM dan akses database dengan Hibernate
MySQL Driver	mysql-connector-j	Driver koneksi ke database MySQL
PostgreSQL Driver	postgresql	Driver koneksi ke database PostgreSQL
Lombok	lombok	Mengurangi boilerplate (getter, setter, dll)
Spring Boot DevTools	spring-boot-devtools	Hot reload saat development

2.3 Struktur Folder Project

```
data/
├── src/
│   ├── main/
│   │   ├── java/com/tup/data/
│   │   │   ├── DataApplication.java      ← Entry point aplikasi
│   │   │   ├── controller/
│   │   │   │   └── UserController.java    ← REST Controller
│   │   │   ├── service/
│   │   │   │   └── UserService.java      ← Business Logic
│   │   │   ├── repository/
│   │   │   │   └── UserRepository.java   ← Database Access
│   │   │   ├── model/
│   │   │   │   └── User.java             ← Entity/Model
│   │   └── resources/
│   │       └── application.properties    ← Konfigurasi
│   └── test/
│       └── application.properties        ← Unit Test
└── pom.xml                             ← Maven dependencies
```

2.4 File pom.xml — Dependensi Maven

```
<dependencies>
  <!-- Spring Web untuk REST API -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Spring Data JPA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- MySQL Driver -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Lombok (opsional) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

3.1 File application.properties

File konfigurasi utama Spring Boot. Terletak di src/main/resources/application.properties.

Konfigurasi MySQL:

```
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/db_data?useSSL=false&serverTimezone=Asia/Jakarta
spring.datasource.username=root
spring.datasource.password=

# MySQL Driver
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Konfigurasi PostgreSQL (Jika pakai postgree SQL):

```
spring.datasource.url=jdbc:postgresql://localhost:5432/db_data
spring.datasource.username=postgres
spring.datasource.password=password_kamu
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

3.2 Penjelasan Parameter Penting

Parameter	Penjelasan
spring.datasource.url	URL koneksi — tentukan host, port, dan nama database
spring.datasource.username	Username database
spring.datasource.password	Password database
spring.jpa.hibernate.ddl-auto	Kontrol bagaimana Hibernate mengelola skema tabel
spring.jpa.show-sql	Tampilkan query SQL di console (true = aktif)
spring.jpa.properties.hibernate.format_sql	Format SQL agar mudah dibaca di console
server.port	Port server (default: 8080)

3.3 Nilai ddl-auto — Penjelasan Detail

Nilai	Kapan Digunakan	Efek
create	Testing awal	DROP semua tabel lalu CREATE ulang — DATA HILANG
create-drop	Unit testing	Sama seperti create, tapi DROP saat app berhenti
update	Development	Update struktur tabel jika ada perubahan, data aman
validate	Staging/Production	Hanya validasi, tidak ubah tabel sama sekali
none	Production	Tidak lakukan apapun — kelola skema secara manual

Peringatan

Jangan gunakan ddl-auto=create atau create-drop di production! Data akan terhapus setiap kali aplikasi restart.

3.4 Connection Pooling dengan HikariCP (Opsional)

Spring Boot secara default menggunakan HikariCP sebagai connection pool — yang terbaik dan tercepat di ekosistem Java.

```
# Konfigurasi HikariCP (opsional, sudah ada default yang bagus)
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```


4.1 Apa itu Entity?

Entity adalah class Java yang merepresentasikan tabel di database. Setiap instance dari class Entity akan menjadi satu baris (row) dalam tabel yang bersangkutan.

4.2 Contoh Entity Lengkap

```
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name="name", nullable = false, length = 100)
    private String name;

    @Column(name = "email", nullable = false, unique = true)
    private String email;

    @Column(name = "phone", length = 15)
    private String phone;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
    }

    @PreUpdate
    public void preUpdate() {
        this.updatedAt = LocalDateTime.now();
    }
}
```

4.3 Anotasi Entity — Referensi Lengkap

Anotasi	Fungsi & Parameter Penting
@Entity	Wajib ada. Menandai class sebagai JPA entity (tabel)
@Table(name)	Menentukan nama tabel. Default = nama class lowercase
@Id	Menandai field sebagai primary key. Wajib ada satu di setiap entity
@GeneratedValue	AUTO = otomatis, IDENTITY = auto_increment DB, SEQUENCE = urutan
@Column	nullable, unique, length, name, columnDefinition, updatable, insertable
@Transient	Field ini TIDAK akan disimpan ke database
@Enumerated	Menyimpan nilai enum ke database (STRING atau ORDINAL)
@Lob	Untuk data besar seperti text panjang atau binary
@PrePersist	Method dipanggil sebelum entity pertama kali disimpan
@PreUpdate	Method dipanggil sebelum entity diupdate

4.4 Relasi Antar Tabel

Anotasi	Contoh Relasi	Penjelasan
@ManyToOne	Banyak User → 1 Role	Field yang menjadi foreign key
@OneToMany	1 User → Banyak Order	Biasanya diikuti mappedBy
@OneToOne	1 User → 1 Profile	Relasi satu ke satu
@ManyToMany	Banyak User → Banyak Role	Butuh tabel pivot/junction

```
// Contoh @ManyToOne
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "role_id") // nama kolom foreign key
private Role role;

// Contoh @OneToMany
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
private List<Order> orders = new ArrayList<>();
```

💡 FetchType — LAZY vs EAGER

LAZY (Direkomendasikan): Data relasi hanya diambil ketika benar-benar diakses. Lebih hemat resource. **EAGER**: Data relasi langsung diambil bersamaan dengan entity utama. Bisa menyebabkan N+1 problem.

5.1 Apa itu Repository?

Repository adalah interface yang menangani operasi akses database. Spring Data JPA akan secara otomatis generate implementasinya — kita tidak perlu menulis kode implementasi sama sekali!

5.2 Hierarki Repository Spring Data

Interface	Method yang Tersedia
Repository<T,ID>	Interface dasar, tidak ada method (marker interface)
CrudRepository<T,ID>	save, findById, findAll, delete, count, exists
PagingAndSortingRepository	findAll(Sort), findAll(Pageable)
JpaRepository<T,ID>	Semua di atas + flush, saveAndFlush, deleteInBatch

✓ Rekomendasi

Gunakan JpaRepository<NamaEntity, TipeId> untuk mendapatkan semua fitur yang dibutuhkan.

5.3 Membuat Repository

```
import com.tup.data.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // SELECT * FROM users WHERE email = ?
    Optional<User> findByEmail(String email);

    // → SELECT * FROM users WHERE name LIKE '%keyword%'
    List<User> findByNameContaining(String keyword);

    // ===== CUSTOM QUERY JPQL =====
    @Query("SELECT u FROM User u WHERE u.phone = :phone")
    Optional<User> findUserByPhone(@Param("phone") String phone);

    // ===== NATIVE QUERY (SQL murni) =====
    @Query(value = "SELECT * FROM users WHERE created_at >= :date", nativeQuery = true)
    List<User> findUsersCreatedAfter(@Param("date") java.time.LocalDateTime date);
}
```

5.4 Keyword Derived Query Methods

Keyword	Method Contoh	SQL yang Dihasilkan
findBy	findByName(name)	WHERE name = ?
And	findByNameAndEmail	WHERE name=? AND email=?
Or	findByNameOrEmail	WHERE name=? OR email=?
Containing	findByNameContaining	WHERE name LIKE '%?%'
StartingWith	findByNameStartingWith	WHERE name LIKE '?%'
GreaterThan	findByAgeGreaterThan	WHERE age > ?
Between	findByAgeBetween	WHERE age BETWEEN ? AND ?
True/False	findByActiveTrue	WHERE active = true
IsNull	findByPhoneIsNull	WHERE phone IS NULL
OrderBy	findByNameOrderByAge	ORDER BY age ASC

6.1 Service Layer

Service layer berisi logika bisnis aplikasi. Ia menjadi perantara antara Controller (yang menerima request) dan Repository (yang mengakses database).

```
import com.tup.data.entity.User;
import com.tup.data.repository.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Service
@Transactional
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Transactional(readOnly = true)
    public List<User> getAllUsers() {
        return this.userRepository.findAll();
    }

    @Transactional(readOnly = true)
    public User getUserById(Long id) {
        return this.userRepository.findById(id).orElseThrow(() -> new
RuntimeException("User tidak ditemukan"));
    }

    public Optional<User> getUserByEmail(String email) {
        return this.userRepository.findByEmail(email);
    }

    public Optional<User> getUserByPhone(String phone) {
        return this.userRepository.findUserByPhone(phone);
    }

    public List<User> getUserByNameContaining(String name) {
        return this.userRepository.findByNameContaining(name);
    }

    public User createUser(User user) {
        return this.userRepository.save(user);
    }

    public User updateUser(Long id, User updatedData) {
        User userExisting = getUserById(id);
        userExisting.setName(updatedData.getName());
    }
}
```

```

        userExisting.setPhone(updatedData.getPhone());
        return this.userRepository.save(userExisting);
    }

    public void deleteUser(Long id) {
        User userExisting=getUserById(id);
        this.userRepository.delete(userExisting);
    }

    public Long countUsers() {
        return this.userRepository.count();
    }

    public List<User> getUsersCreatedAfter(LocalDateTime date) {
        return userRepository.findUsersCreatedAfter(date);
    }
}

```

6.2 Controller Layer

Controller menerima HTTP request, memanggil service yang sesuai, lalu mengembalikan HTTP response.

```

import com.tup.data.entity.User;
import com.tup.data.service.UserService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserRestController {

    private final UserService userService;

    public UserRestController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        return ResponseEntity.ok(userService.getAllUsers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        User userCreated = userService.createUser(user);
    }
}

```

```

        return
        ResponseEntity.status(HttpStatus.CREATED).body(userCreated);
    }

    @PutMapping("/{id}")
    public ResponseEntity<User> updateUser(@PathVariable Long id,
@RequestBody User user) {
        return ResponseEntity.ok(userService.updateUser(id, user));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<User> deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/count")
    public ResponseEntity<Long> countUsers() {
        return ResponseEntity.ok(userService.countUsers());
    }

    @GetMapping("/email/{email}")
    public ResponseEntity<User> getUserByEmail(@PathVariable String
email) {
        return userService.getUserByEmail(email)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/phone/{phone}")
    public ResponseEntity<User> getUserByPhone(@PathVariable String
phone) {
        return userService.getUserByPhone(phone)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/created-after")
    public ResponseEntity<List<User>>
getUsersCreatedAfter(@RequestParam LocalDateTime date) {
        List<User> users = userService.getUsersCreatedAfter(date);
        return ResponseEntity.ok(users);
    }

    @GetMapping("/namecontain/{name}")
    public ResponseEntity<List<User>>
getUserByNameContaining(@PathVariable String name) {
        return
        ResponseEntity.ok(userService.getUserByNameContaining(name));
    }
}

```

6.3 Anotasi Controller — Referensi

Anotasi	Fungsi
@RestController	Gabungan @Controller + @ResponseBody. Semua return value jadi JSON
@RequestMapping	Definisikan base path untuk class atau method
@GetMapping	Handle HTTP GET request
@PostMapping	Handle HTTP POST request
@PutMapping	Handle HTTP PUT request (update keseluruhan)
@PatchMapping	Handle HTTP PATCH request (update sebagian)
@DeleteMapping	Handle HTTP DELETE request
@PathVariable	Ambil nilai dari URL path: /users/{id}
@RequestParam	Ambil nilai query parameter: /users?name=John
@RequestBody	Ambil data dari body request (JSON)
ResponseEntity<T>	Membungkus response dengan HTTP status code

7.1 Endpoint yang Tersedia

Method	URL	Body	Fungsi
GET	/api/users	-	Ambil semua user
GET	/api/users/{id}	-	Ambil user by ID
POST	/api/users	JSON User	Tambah user baru
PUT	/api/users/{id}	JSON User	Update user
DELETE	/api/users/{id}	-	Hapus user

7.2 Contoh Request & Response

POST /api/users — Membuat User Baru

Request Body (JSON):

```
{
  "name": "Budi Santoso",
  "email": "budi@tup.com",
  "phone": "081234567890"
}
```

Response (201 Created):

```
{
  "id": 1,
  "name": "Budi Santoso",
  "email": "budi@tup.com",
  "phone": "081234567890",
  "active": true,
  "createdAt": "2025-06-02T10:30:00"
}
```

GET /api/users — Ambil Semua User

Response (200 OK):

```
[
  { "id": 1, "name": "Budi Santoso", "email": "budi@example.com" },
  { "id": 2, "name": "Siti Rahayu", "email": "siti@example.com" }
]
```

7.3 HTTP Status Code yang Umum

Kode	Nama	Kapan Digunakan
200 OK	OK	Request berhasil (GET, PUT, DELETE)
201 Created	Created	Data berhasil dibuat (POST)
204 No Content	No Content	Berhasil tapi tidak ada response body (DELETE)
400 Bad Request	Bad Request	Input/request tidak valid
404 Not Found	Not Found	Data yang dicari tidak ada
500 Internal Error	Server Error	Ada error tidak terduga di server

8.1 Tabel Error dan Solusi

Error Message	Penyebab	Solusi
Communications link failure	MySQL tidak berjalan / salah port	Start MySQL service, cek port 3306
Access denied for user	Username/password salah	Cek application.properties
Unknown database 'nama_db'	Database belum dibuat	CREATE DATABASE nama_db di MySQL
Table 'x' doesn't exist	Tabel belum ada	Ganti ddl-auto ke update atau create
LazyInitializationException	Akses data relasi di luar transaksi	Tambah @Transactional di service
No qualifying bean found	Class tidak dideteksi Spring	Cek @Repository, @Service, @Component
Port 8080 already in use	Port sudah dipakai proses lain	Ganti server.port=8081 di properties
Could not autowire: no beans	Dependency injection gagal	Pastikan anotasi & scan package benar

8.2 Checklist Debug

- Pastikan database server (MySQL/PostgreSQL) sudah berjalan
- Cek nama database sudah dibuat di database server
- Cek kesesuaian username, password, dan URL di application.properties
- Aktifkan show-sql=true untuk melihat query yang dijalankan
- Baca pesan error di console dari baris PALING BAWAH (root cause)
- Pastikan semua anotasi (@Entity, @Repository, @Service) terpasang
- Restart aplikasi setelah mengubah application.properties

Tips: Baca Log dengan Benar

Spring Boot menampilkan stack trace dari atas ke bawah. Error sebenarnya ada di bagian PALING BAWAH (Caused by: ...). Mulai baca dari sana, bukan dari atas!



Checklist Kompetensi

✓	Kompetensi yang Harus dikuasai
☐	Mampu membuat project Spring Boot baru via Spring Initializr dengan dependensi yang tepat
☐	Memahami struktur folder dan peran setiap layer (Controller, Service, Repository, Entity)
☐	Mampu mengkonfigurasi koneksi database di application.properties dengan benar
☐	Memahami perbedaan nilai ddl-auto dan kapan menggunakannya
☐	Mampu membuat Entity class dengan anotasi JPA yang tepat
☐	Mampu membuat Repository dengan derived query methods dan @Query custom
☐	Mampu membuat Service layer dengan validasi bisnis dan @Transactional
☐	Mampu membuat REST Controller dengan operasi CRUD lengkap
☐	Mampu menguji API dengan Postman dan memahami HTTP status code
☐	Mampu mendiagnosis error umum koneksi database dan menemukan solusinya

Latihan Mandiri

Setelah memahami materi, coba implementasikan project berikut secara mandiri:

Tugas: CRUD Produk

Buat aplikasi manajemen produk sederhana dengan field:

- id (Long, auto increment)
- name (String, tidak boleh null)
- price (Double, tidak boleh null)
- stock (Integer, default 0)
- category (String)
- createdAt (LocalDateTime, auto-fill)
- updatedAt (LocalDateTime, auto-fill)

Buat endpoint:

- GET /api/products — ambil semua produk
- GET /api/products/{id} — ambil produk by ID
- GET /api/products/category/{cat} — filter by category
- POST /api/products — tambah produk baru
- PUT /api/products/{id} — update produk
- DELETE /api/products/{id} — hapus produk

Referensi & Sumber Belajar Lanjutan

Sumber	URL / Keterangan
Dokumentasi Resmi	docs.spring.io/spring-data/jpa/docs/current/reference
Spring Initializr	https://start.spring.io
Hibernate Docs	hibernate.org/orm/documentation
Baeldung	baeldung.com/spring-data-jpa-tutorial
YouTube Spring	Amigoscode, Dan Vega, Java Brains
Postman	postman.com — tool testing REST API

Catatan